

BinHTTP/1 - Protocol specification

Revision 1.2 | Track 1: server | Page 1 of 2

MUST/MUST NOT are requirements. All integers are unsigned, big-endian. Lengths count bytes, never characters. Text is strict UTF-8 without terminators, except literal header names (ASCII). There is no handshake or magic prefix: the first byte is part of the first frame.

1. Connection and ordering

Use one TCP connection for multiple exchanges. A request is one REQUEST frame; its response is one RESPONSE frame. Read exactly the 8-byte frame header, then exactly Length payload bytes. TCP reads may return partial frames or multiple frames together. EOF is not a message delimiter.

The server MUST keep the connection open after complete responses unless CLOSE is requested or a fatal error occurs. It handles requests and returns responses in arrival order, including pipelined requests. IDs are 1-65535, copied into responses; clients MUST NOT reuse an outstanding ID. After 65535, reuse 1 once it is free. Clients match each response to the oldest outstanding request. ID mismatch or a malformed response is fatal; close without retrying. Unexpected EOF fails outstanding requests. This server admits at most 64 simultaneous connections; excess sockets close before any frame is read, without affecting admitted connections.

2. Fixed frame header

Offset	Bytes	Field	Meaning
0	4	Length	Payload byte count; excludes this header
4	1	Type	0x01 REQUEST, 0x02 RESPONSE
5	1	Flags	Bit 0 is CLOSE; other bits reserved
6	2	Request ID	Echoed in the response; 0 reserved

Why 32/8/8/16 bits? Whole files occupy single frames: 32-bit lengths permit up to 4,294,967,295 payload bytes, including response metadata. Eight-bit types and flags leave extension space. Sixteen-bit IDs suffice for ordered exchanges with reuse. HTTP/2 uses 24/8/8/31 bits (plus a reserved bit): it splits bodies across frames and multiplexes streams whose IDs cannot be reused. This protocol needs neither multiplexing nor a 31-bit stream ID.

3. Payloads

3.1 REQUEST (Type 0x01)

Bytes	Field	Rule
1	Method	0x01 = GET; no other method defined
2	Path length	N, the UTF-8 byte count
N	Path	Nonempty; starts with /; see section 7
1	Header count	H, from 0 through 255
Variable	Headers	Exactly H entries using section 6

GET has no body. The payload MUST end immediately after its last header. A REQUEST payload MUST NOT exceed 65,536 bytes. No request headers are mandatory.

3.2 RESPONSE (Type 0x02)

The payload is Status (2 bytes), Header count (1 byte), that many headers (section 6), then the raw body bytes. The body occupies all remaining payload bytes, including zero bytes for an empty file. Body size is Length minus the status, count and encoded headers. No chunking or extra terminator is used.

4. Flags and closing

On REQUEST, CLOSE (0x01) asks the server to send one complete response with CLOSE set, then close. This also applies to error responses. On RESPONSE, CLOSE tells the client to stop sending and close; other outstanding requests fail. Senders MUST clear reserved flag bits; receivers MUST ignore them. Idle connections remain open until peer closure or server shutdown; there is no idle timeout.

5. Unknown frame types - required extension rule

The server acts only on REQUEST; the client acts only on RESPONSE. For EVERY other type, the receiver MUST discard exactly Length bytes, send no reply, and continue at the next frame. Ignore its flags and ID, including CLOSE. Unknown frames are not subject to the REQUEST size limit and are discarded without allocating their entire payload. EOF while skipping is fatal. This rule lets a future version add types without losing frame boundaries.

BinHTTP/1 - Headers and server behaviour

Revision 1.2 | Track 1: server | Page 2 of 2

6. Header encoding

Each entry is Name ID (1 byte), optionally Name length (1 byte) and Name, then Value length (2 bytes) and Value. With ID 0, the literal name MUST contain 1-255 lowercase ASCII token bytes: a-z, 0-9, and !#\$%&'+-.^_`~. With IDs 1-10, omit the name length and name; use the table. Value length is 0-65535 UTF-8 bytes. Names MUST NOT repeat, including a literal spelling of an indexed name.

ID	Name	Typical sender/value
1	host	Client: localhost:9000
2	user-agent	Client: spec-probe/1
3	accept	Client: */*
4	server	Server: bserve/1.1
5	date	Server: current IMF-fixdate in GMT
6	content-type	Server: text/plain, text/html, etc.
7	content-length	Either: body byte count in decimal ASCII
8	last-modified	Server: file mtime as IMF-fixdate
9	etag	Server: W/"size-mtime", both hexadecimal
10	cache-control	Server: max-age=60

These ten names cover the three usual request headers and seven successful-response headers. Name IDs save repeated text; length-prefixed literals support other names. Unlike HPACK, there is no dynamic table or Huffman coding. IDs 11-255 are reserved: read Value length, skip exactly that many opaque bytes without decoding, and ignore the entry.

When present, content-length MUST be nonempty ASCII digits and equal the actual body size; leading zeros are allowed. GET has no body, so its value may contain only zeros. This header never changes framing. Dates look like Thu, 08 Oct 2026 12:00:00 GMT. The size/mtime ETag is weak because it is not a content hash. Headers are descriptive; conditional requests and caching logic are not defined.

7. Paths, errors and limits

Paths are literal UTF-8 file paths using / separators. They MUST NOT contain NUL, backslash or colon, or start with //. No URL decoding, query parsing or fragment parsing occurs: %, ? and # are literal filename characters where the host filesystem permits them. Append index.html to paths ending in /. Strip the leading /, resolve beneath the canonical root, and normalize . and .. BEFORE any filesystem lookup. Reject an escape with 403. Resolve symlinks on that normalized path and reject existing targets outside the canonical root with 403. Thus /missing/./hello.txt maps to /hello.txt on every OS. Do not serve directories. Missing targets (including broken symlinks) return 404.

Status	Meaning
200	Regular file found; body contains its exact bytes
400	Malformed REQUEST, as defined below
403	Path escapes the document root
404	No regular file at the requested path
500	Filesystem/read failure, or response cannot fit in a frame

A complete REQUEST is malformed if any field overruns its payload, bytes remain after the headers, Method is not 1, ID is 0, path violates its rules, literal name is empty/invalid, known header text is invalid UTF-8, a name repeats, or content-length is invalid. Reply 400 with the same ID (even 0) and continue, unless CLOSE was requested. Invalid known response fields, duplicate names or mismatched content-length are fatal to clients. Clients treat 200-399 as success and all other status codes as failure; error status alone does not terminate the connection.

An oversized REQUEST is exceptional: after its 8-byte header, reply 400 with CLOSE and close without reading its payload. EOF inside a header/payload closes the connection without a response. Filesystem errors before sending a response receive 500; an I/O failure after a response starts closes the connection, never inserts another frame into its body. Files must remain stable while served. Responses stream exactly the announced file size; file shrinkage causes a fatal truncated response. Errors carry a short text/plain body; server, date and content-length are included.

8. Minimal request example

00 00 00 0e 01 00 00 01 01 00 0a 2f 68 65 6c 6c 6f 2e 74 78 74 00

This is Length=14, Type=REQUEST, Flags=0, ID=1, Method=GET, Path length=10, Path=/hello.txt, Header count=0. Total: 8 + 14 = 22 bytes. The connection stays open. See HEXDUMP.md for an annotated captured exchange, including response headers and body.